Network Swiss Army Knife (NSAK)

Bachelor Thesis – Integrating Agentic AI for Network Reconnaissance

June 16, 2026

Lukas von Allmen and Frank Gauss | Bern University of Applied Sciences

Overview

- Introduction
- Implementation
- Evaluation Setup
- Ai Integration
- Results
- Discussion and Conclusion
- Demo: Agentic Reconnaissance in Action

Introduction

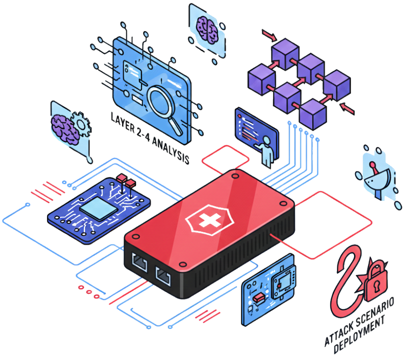
Starting Point: The NSAK Framework

Established in the predecessor project (Project 2)

A modular, open-source network security framework

Built around four reusable resource types:

- **Device** *execution host at a network position*
- **Environment** *target network topology*
- **Scenario** *a red/blue team use case*
- **Drill** *reusable building block*



- Gather baseline data
- Detect
- Alert
- Investigate
- Plan a response
- Execute

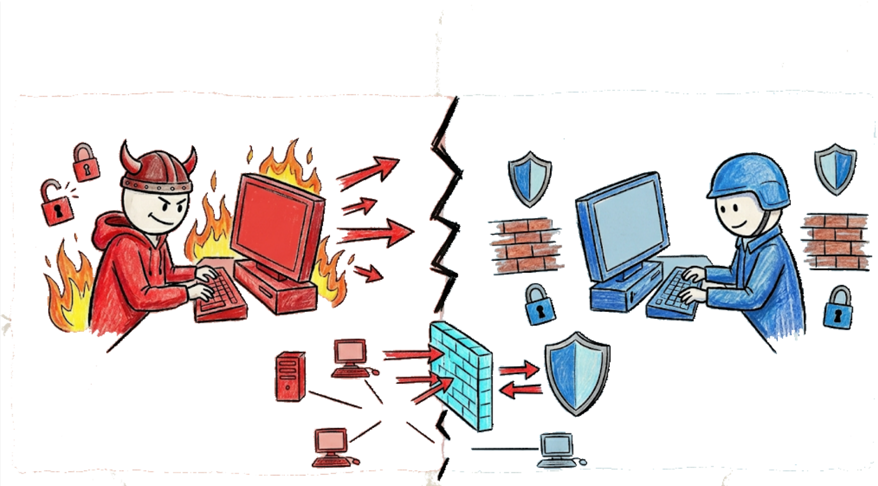


Figure: Red and Blue Team

Reconnaissance

The first stage of the cyber kill chain

Gathering information about a target network *before* any attack – it builds the map every later stage relies on, and is the focus of this thesis.

- **Host discovery** *which systems are reachable on the network*
- **Service & port enumeration** *what is running and exposed*
- **Characterization** *OS, versions and vulnerability indicators*

Hypothesis and Research Questions

From static scenarios to agentic AI

Hypothesis

Integrating **agentic AI** into NSAK improves **adaptability, flexibility** and **automation** over static scenarios, while **lowering the operator's required expertise** to interpret results and plan follow-up steps.

Hypothesis and Research Questions

From static scenarios to agentic AI

Hypothesis

Integrating **agentic AI** into NSAK improves **adaptability, flexibility** and **automation** over static scenarios, while **lowering the operator's required expertise** to interpret results and plan follow-up steps.

RQ1 How does an agentic AI scenario perform **compared to a static reference** scenario in network reconnaissance?

RQ2 To what extent can agentic AI **support operators** during interactive reconnaissance?

RQ3 To what extent is the implementation **suitable for real-world deployment**?

Project Goals

Order of implementation: Must → Should → Could

Must

- Compare Frameworks
- AI Integration Research
- Configuration Management
- AI Scenarios
- Reproducible Test Env.
- Red Team Reconnaissance
- AI Scenario Evaluation
- AI vs. Engineered (Red)

Project Goals

Order of implementation: Must → Should → Could

Must

- Compare Frameworks
- AI Integration Research
- Configuration Management
- AI Scenarios
- Reproducible Test Env.
- Red Team Reconnaissance
- AI Scenario Evaluation
- AI vs. Engineered (Red)

Should

- Blue Team Intrusion Detection
- AI vs. Engineered (Blue)

Project Goals

Order of implementation: Must → Should → Could

Must

Compare Frameworks
AI Integration Research
Configuration Management
AI Scenarios
Reproducible Test Env.
Red Team Reconnaissance
AI Scenario Evaluation
AI vs. Engineered (Red)

Should

Blue Team Intrusion Detection
AI vs. Engineered (Blue)

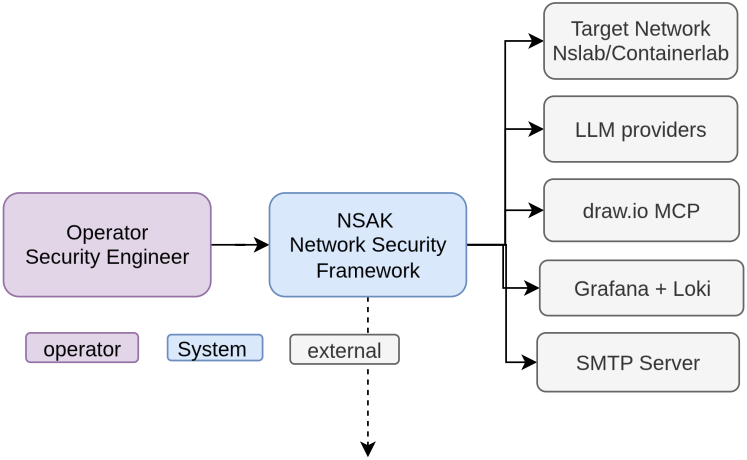
Could

REST API
MCP Server
Web GUI

Implementation

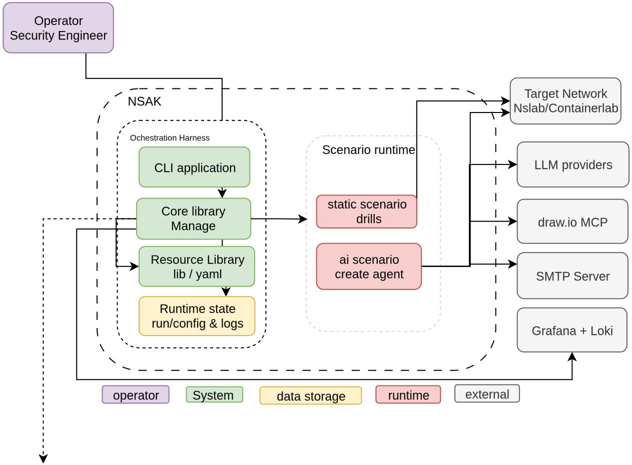
Layered Architecture

system-diagram



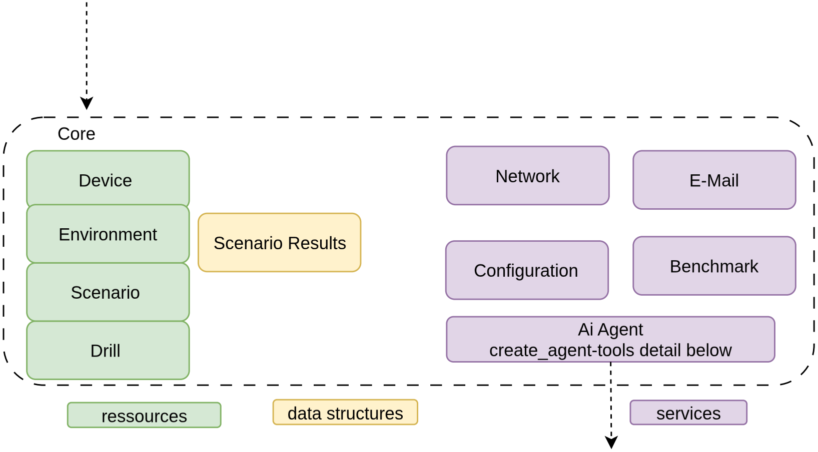
Layered Architecture

Lean Component Diagram



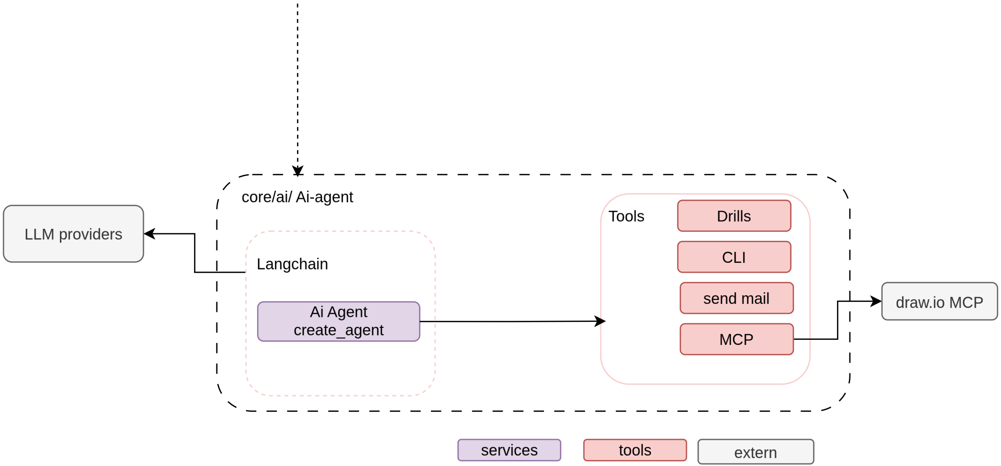
Layered Architecture

Core Diagram and Agent



Layered Architecture

Core Diagram and Agent



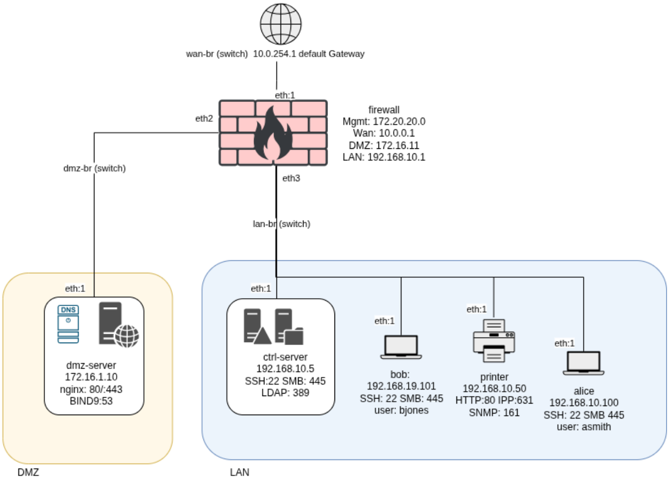
Evaluation Setup

Methology

3 LLMs • 2 Network Environments • Static Reference Baseline

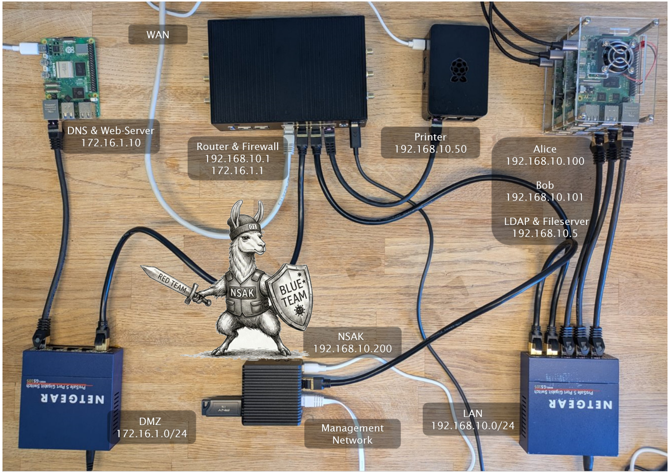
Reproducible Test Environment

A virtual enterprise network built with Containerlab



Physical Lab

The same network implemented with real hardware



Benchmarking Suite

Automated test runs for non-interactive scenarios

Ai Integration

1. Provider Model und Agent
2. Tool calling (Langchain Tools) (MCP) (human interaction hook) (cli) (draw io) (send-mail)
3. 1. Single Agent interactive unstructured output
4. 2. Single Agent Reconnaissance unstructured output
5. 3. POC Blueteam intrusion detection - not followed time and scope - unstructured output
6. 4. Single Agent reconnaissance structure output (context-window, grammar -> why structured output create larger grammar)
7. 5. Multi Agent reconnaissance structured output (issue with smaller models) () Containerlab evalutation smaller model
8. 6. Multi Agent reconnaissance unstructured output

Concept 1: Taming the Context

From one overloaded agent to a focused pipeline

⚠ The problem

- One agent does everything in a single long conversation
- Every scan output piles up → the **context keeps growing**
- Smaller models lose the thread and start failing
- Cost and runtime become hard to predict

Concept 1: Taming the Context

From one overloaded agent to a focused pipeline

The problem

- One agent does everything in a single long conversation
- Every scan output piles up → the **context keeps growing**
- Smaller models lose the thread and start failing
- Cost and runtime become hard to predict

Our solution

- Split the task into **discovery** → **enumeration** → **assessment**
- Each phase is a **fresh agent** with its own, small context
- We pass on only the **distilled result**, not the whole history

Concept 1: Taming the Context

From one overloaded agent to a focused pipeline

The problem

- One agent does everything in a single long conversation
- Every scan output piles up → the **context keeps growing**
- Smaller models lose the thread and start failing
- Cost and runtime become hard to predict

Our solution

- Split the task into **discovery** → **enumeration** → **assessment**
- Each phase is a **fresh agent** with its own, small context
- We pass on only the **distilled result**, not the whole history

Trade-off: rescues small models, but adds orchestration overhead that frontier models don't need.

Concept 2: Keeping the Operator in Control

The agent proposes – the human decides on risky actions

⚠ The problem

- An autonomous agent can run **any** command or send reports on its own
- In a security context that is unacceptable without oversight

Concept 2: Keeping the Operator in Control

The agent proposes – the human decides on risky actions

⚠ The problem

- An autonomous agent can run **any** command or send reports on its own
- In a security context that is unacceptable without oversight

Concept 2: Keeping the Operator in Control

The agent proposes – the human decides on risky actions

The problem

- An autonomous agent can run **any** command or send reports on its own
- In a security context that is unacceptable without oversight

Our solution

- The agent **pauses** before sensitive actions and asks for a decision
- Shell commands → **approve** before they run
- Sending a report → **approve / reject**
- Open questions → operator **responds** in free text

Concept 2: Keeping the Operator in Control

The agent proposes – the human decides on risky actions

The problem

- An autonomous agent can run **any** command or send reports on its own
- In a security context that is unacceptable without oversight

Our solution

- The agent **pauses** before sensitive actions and asks for a decision
- Shell commands → **approve** before they run
- Sending a report → **approve / reject**
- Open questions → operator **responds** in free text

After the decision the agent resumes exactly where it stopped – no restart, no lost progress.

Single Agent

- One prompt to achieve the goal
- Long chain of tool calls, **growing context**
- Smaller models fail as context fills up

Single Agent

- One prompt to achieve the goal
- Long chain of tool calls, **growing context**
- Smaller models fail as context fills up

Multi Agent *divide-and-conquer*

- Prompt split into steps; result fed into next agent
- Smaller, focused context per step
- Helps small models; may add overhead for frontier

Single Agent

- One prompt to achieve the goal
- Long chain of tool calls, **growing context**
- Smaller models fail as context fills up

Multi Agent *divide-and-conquer*

- Prompt split into steps; result fed into next agent
- Smaller, focused context per step
- Helps small models; may add overhead for frontier

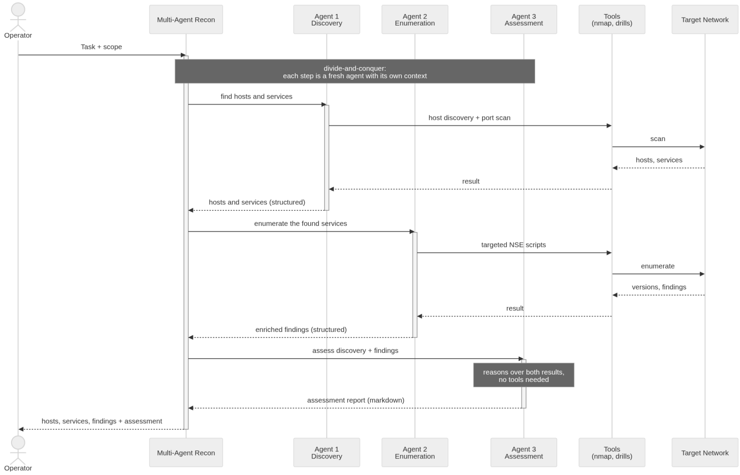
Structured vs. Unstructured output

Structured (schema-constrained): great for automated metrics, but **high failure rate** – especially for small models.

Unstructured: needed in the BFH lab where all models failed to produce valid structured output; freed the agent but required manual evaluation.

Multi-Agent Reconnaissance Pipeline

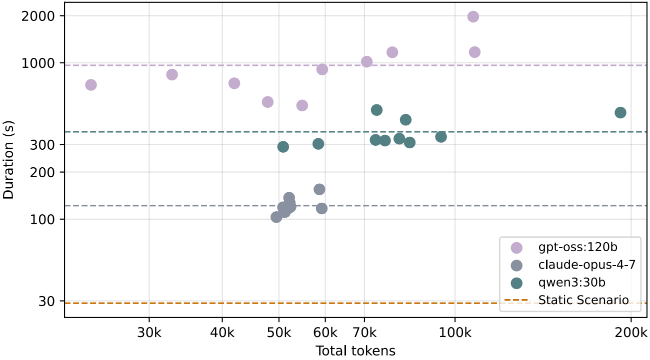
Discovery → enumeration → assessment, each a fresh agent



Results

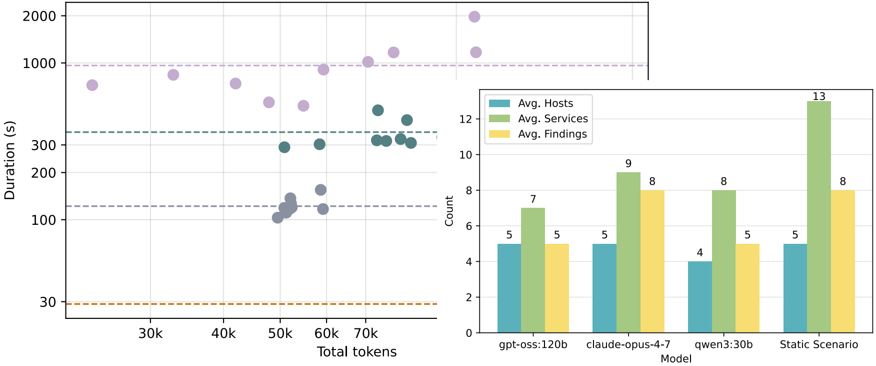
Containerlab – Multi-Agent Benchmark

Containerlab Findings



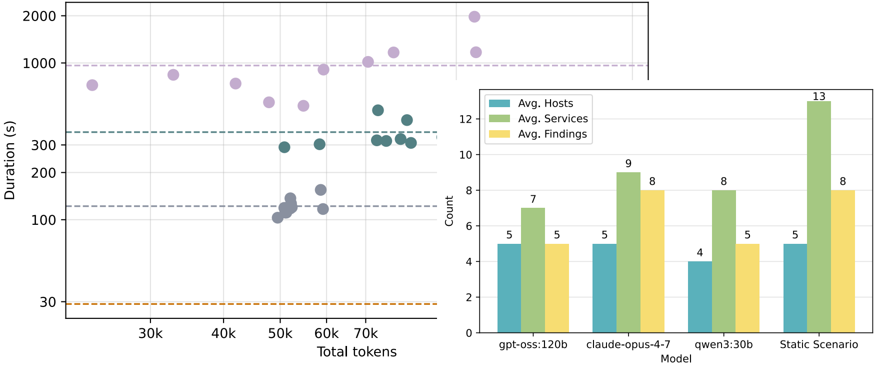
Containerlab – Multi-Agent Benchmark

Containerlab Findings



Containerlab – Multi-Agent Benchmark

Containerlab Findings



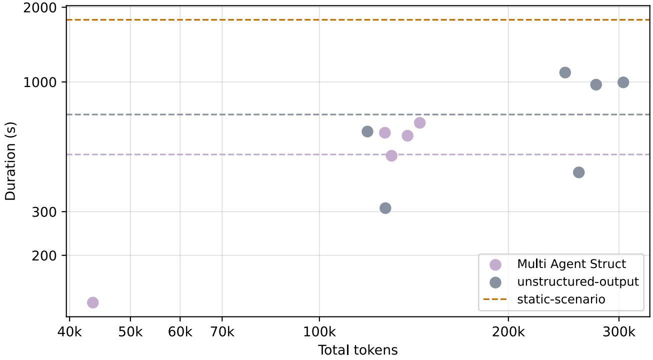
- Decomposition: `qwen3:30b` drops tokens **−68%** and duration **−30%**.
- `claude-opus-4-7` keeps identical discovery quality (slight +13% token overhead).

single agent resultate -> Lösung zu Multiagent Structured vs Unstructured interactive

Qualitative + Interactive

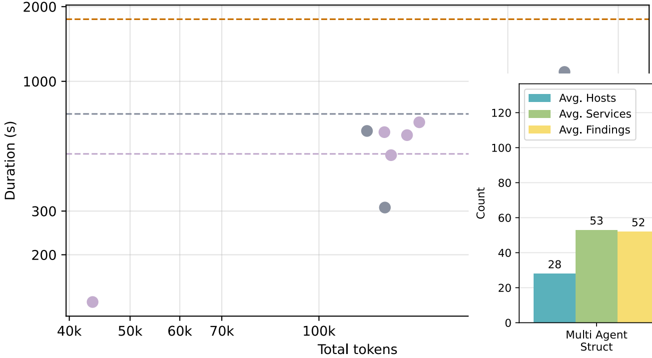
Network Security Lab – Multi-Agent Benchmark

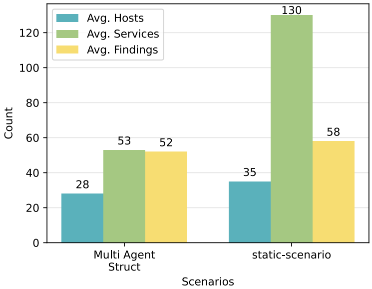
NSlab - Findings



Network Security Lab – Multi-Agent Benchmark

NSlab - Findings





Discussion and Conclusion

Key Findings

Takeaways across both environments

What works

- Even smaller models can autonomously run **standard red-team reconnaissance**
- **Multi-agent decomposition** helps smaller models (–68% tokens for `qwen3:30b`)
- AI delivers **depth**: assessment, prioritized findings, remediation

Key Findings

Takeaways across both environments

What works

- Even smaller models can autonomously run **standard red-team reconnaissance**
- **Multi-agent decomposition** helps smaller models (–68% tokens for `qwen3:30b`)
- AI delivers **depth**: assessment, prioritized findings, remediation

What limits it

- **High failure rate** – best-case subset
- **Hallucinations**: fabricated CVEs, honeypots

Answering the Research Questions

RQ1 **AI vs. static:** Same discovery power as baseline, no gains in speed/determinism. Adds value via **reasoning and interpretation** (credentials, banner reasoning, severity scoring).

Answering the Research Questions

RQ1 **AI vs. static:** Same discovery power as baseline, no gains in speed/determinism. Adds value via **reasoning and interpretation** (credentials, banner reasoning, severity scoring).

RQ2 **Operator support:** Converts scans into structured, severity-ranked output + next steps. `--interactive` Operator stays in the loop and control.

Answering the Research Questions

RQ1 **AI vs. static:** Same discovery power as baseline, no gains in speed/determinism. Adds value via **reasoning and interpretation** (credentials, banner reasoning, severity scoring).

RQ2 **Operator support:** Converts scans into structured, severity-ranked output + next steps. **`--interactive`** Operator stays in the loop and control.

RQ3 **Deployment:** Scales to ∼35 hosts, highlights key risks. **Requires supervision** (reliability limits).

Project Goals

Order of implementation: Must → Should → Could

Must

- Compare Frameworks
- AI Integration Research
- Configuration Management
- AI Scenarios
- Reproducible Test Env.
- Red Team Reconnaissance
- AI Scenario Evaluation
- AI vs. Engineered (Red)

Should

- Blue Team Intrusion Detection
- AI vs. Engineered (Blue)

Could

- REST API
- MCP Server
- Web GUI

Project Goals

Order of implementation: Must → Should → Could

Must

- Compare Frameworks ✓
- AI Integration Research ✓
- Configuration Management ✓
- AI Scenarios ✓
- Reproducible Test Env. ✓
- Red Team Reconnaissance ✓
- AI Scenario Evaluation ✓
- AI vs. Engineered (Red) ✓

Should

- Blue Team Intrusion Detection
- AI vs. Engineered (Blue)

Could

- REST API
- MCP Server
- Web GUI

Recommendations and Future Work

Recommendations

- **Match model tier to task** – frontier model for autonomous recon
- **Multi-agent** for smaller / self-hosted models
- **Hybrid pipeline**: static breadth + AI depth (+ RAG)
- **Human-in-the-loop** + retry on incomplete runs

Future work

- **Dynamic tool calling** – expose only relevant tools
- **Guardrails & sandboxing** – scope restriction for production
- Robust **HITL middleware**
- **Broader attack-chain** coverage beyond reconnaissance

Conclusion

Agentic AI complements deterministic enumeration

- ✔ Using NSAK directly as an **agentic harness** proved effective – the agent autonomously and interactively executes network discovery via LangChain tools.
- ✔ Scripted scenarios excel in correctness and completeness; the agent adds **reasoning and assessment**, limited by model capability.
- ✔ **Multi-agent decomposition** models to digest large context improves models by reducing context complexity and increasing consistency.
- ⚠ Hallucinations remain a limitation – **supervised operation with a human reviewer** is the proposed deployment model.

LLMs make cybersecurity use cases more accessible and cost-effective – with a human in the loop.

Personal Conclusion

Frank

-

Lukas

- Initially skeptical of the usefulness of AI
- We covered a lot of topics we learned at BFH
- A bit of naivety in the planning led to a larger scope
- Keep focusing on the goals and research questions
- AI is here to stay as another engineering tool

Demo: Agentic Reconnaissance in Action